

# n-grams counting

**Edoardo Sarri**

Parallel Computing

Febbraio 2026



UNIVERSITÀ  
DEGLI STUDI  
FIRENZE

# Introduzione

Conteggio degli  $n$ -grams (sequenze di  $n$  parole consecutive) in file di testo.

- Input:
  - File di testo inglese (normalizzazione semplificata).
  - Lunghezza  $n$  degli  $n$ -grams (default 3).
- Output: Statistiche sugli  $n$ -grams più frequenti e sui tempi di esecuzione.

## Versioni

Sono state realizzate due implementazioni:

- Sequenziale (C).
- Parallela (OpenMP).

# Introduzione

## Input

Dataset testuale (es. **Hugging Face C4**).

- Script Python per il download. Permette di specificare la dimensione del file di input.

## Normalizzazione

Creazione di classi di equivalenza per parole simili:

- Tutto in minuscolo.
- Rimozione punteggiatura.
- Si mantengono solo caratteri alfanumerici e singolo spazio.

# Introduzione

## Struttura Dati

Utilizzo di una Hash Table per contare gli  $n$ -grams.

- Hash Function: Polynomial Hash,

$$H(c) = \left( \sum_{i=1}^n c_i \cdot p^{n-i} \right) \mod M$$

- Hash Function implementation: Dato  $H(c_1 \cdots c_i)$ ,

$$H(c_1 \cdots c_i c_{i+1}) = (H(c_1 \cdots c_i) \cdot p + c_{i+1}) \mod M.$$

- Collisioni: Open Chain.

# Introduzione

## Struttura Dati

Hash Function:  $H(c) = \left( \sum_{i=1}^n c_i \cdot p^{n-i} \right) \bmod M$

- $p$ : Deve essere primo e più grande della dimensione dell'alfabeto. Abbiamo scelto  $p = 257$ .
- $M$ : Deve essere primo. Regolare  $M$  permette di ottenere un fill factor  $\alpha = \frac{\text{busy\_buckets}}{\text{table\_length}}$  migliore o peggiore. Abbiamo scelto  $M = 10000019$ .
- Overflow: Problemi se durante il calcolo  $H(c_1, \dots, c_i)$  è più grande del tipo di dato usato per un qualche  $i$ . Abbiamo usato il tipo più veloce per l'architettura di almeno 32 bit (i.e., `uint_fast32_t`):  $\text{inter\_var\_dim} = (M - 1) \cdot p + c_{\max} < 2^{32}$ .

# Sequential

L'**implementazione sequenziale** segue i seguenti step:

- 1 **Data Loading:** In **map\_file()** si carica il file di input.
- 2 **N-gram Extraction:** Buffer circolare si dimensione  $n$  per mantenere l' $n$ -gram corrente.
- 3 **Populate Hash Table:** Inserimento/aggiornamento con **add\_gram()** del contatore per ogni  $n$ -gram incontrato.
- 4 **Reporting:** Calcolo e stampa delle **statistiche** finali.

# Sequential

## Data Loading

Il file di input può raggiungere grandi dimensioni.

- **Problema:** Overhead per copie multiple e system calls se facciamo molte `read()`.
- **Desiderato:** Sarebbe comodo trattare l'input come un singolo array di caratteri.
- **Soluzione:** Memory Mapped I/O con `mmap`.
- **Ottimizzazione:** Si indica al kernel l'accesso sequenziale al file con `madvise` e il flag `MADV_SEQUENTIAL`.

# Sequential

## Allocazione

Dobbiamo memorizzare ogni  $n$ -gram incontrato.

- **Problema:** Due problemi:
  - Una `malloc` alloca sia lo spazio richiesto (per noi poco) che metadati: i metadati non sono irrilevanti rispetto alla dimensione di un  $n$ -gram.
  - Con più thread, `malloc` richiede un system lock: troppo overhead.
- **Soluzione:** Usare un *Arena Allocator*. Si alloca più memoria contigua (16 MB di default) e si prende via via quello che ci serve.



# Sequential

## Allocazione

Dobbiamo memorizzare ogni  $n$ -gram incontrato.

- **Vantaggi:** Poche `malloc`. Si riduce la frammentazione e si migliora la località spaziale.
- **Quantificazione:**
  - Popolazione: Nella versione con molte `malloc` il popolamento della hash table richiede  $\approx 18s$ , con l'arena allocator si scende a  $\approx 10s$ .
  - Free: Si passa da  $\approx 2s$  a  $\approx 0.001s$ .
- **Profiling:** Nella prima versione la maggior parte del tempo è speso in `malloc`, mentre con l'arena allocator il tempo è quasi interamente nella funzione `add_gram()`.

# Sequential

## Statistiche

Per l'estrazione delle statistiche sono stati valutati due approcci: sorting e top-K.

## Statistiche - Sorting

- Si definisce un array di puntatori di lunghezza  $N$ , con  $N$  numero di  $n$ -grams unici.
- Popolare questo array con i puntatori agli  $n$ -grams nella hash table.
- Ordinare l'array sulla frequenza.
- Stampare i primi  $K$ .

# Sequential

## Statistiche - Top-K (implementata)

- Si definisce un array di puntatori di lunghezza  $K$ .
- Iterazione singola sulla Hash Table, sostituendo il minimo se si trova un  $n$ -gram più frequente.

# Sequential

Per l'estrazione sono stati valutati due approcci.

## Statistiche - Complessità

### ■ Tempo:

- Sorting: Si deve passare sulla hash table e poi ordinare e quindi  $O(M + N \log N)$ .
- Top-K: Si passa sulla hash table una sola volta e per ogni suo elemento si scandisce la tabella dei top-K, quindi  $O(M \cdot K) \approx O(M)$ .

### ■ Spazio:

- Sorting:  $O(N)$
- Top-K:  $O(K)$

# Parallel

L'**implementazione parallela** segue il pattern Map-Reduce secondo i seguenti step:

- 1 Inizializzazione:** Prima della parallelizzazione, il main thread alloca la tabella globale e quelle locali.
- 2 Popolazione**
- 3 Merge**
- 4 Statistiche:** Vengono estratte in modo sequenziale.

# Parallel

## Inizializzazione

- Tabella globale: Stessa dimensione della versione sequenziale.
- Vettore di tabelle locali: Serve per gestire a fase di merge. Ha tanti elementi quanti sono i thread.
- Tabelle locali: Per semplificare la fase di merge, ha la stessa dimensione di quella globale.
- Indici: Ogni thread ha il suo indice di inizio e fine del chunk del file di input.
- Variabili in `populate_hashtable()`:
  - Sono `shared` la tabella globale e il vettore di tabelle locali.
  - Sono `firstprivate` le variabili di inizio e fine del chunk.

# Parallel

## Popolazione (Map)

- Parallelizzazione: Si usa `# pragma omp parallel` e non `parallel for` per gestire meglio i boundaries e l'allignment.
- Boundaries: Un  $n$ -gram è processato dal thread  $i$ -esimo se la sua prima parola è nel chunk assegnato a tale thread.
- Allignment: Se l'indice di inizio è a metà parola, allora si avanza all'inizio della parola successiva. Non fatto per il main thread.
- Arena: Ogni thread ha il proprio arena allocator per evitare di usare lock quando si richiede spazio.

# Parallel

## Merge (Reduce)

- Parallelizzazione non su thread ma su bucket: Ogni thread lavora sulla stessa partizione della tabella globale e ogni tabella locale. Permette di non usare mutex.
- Zero-copy: Se un  $n$ -gram è presenta nella tabella globale allora si aggiorna il contatore, altrimenti si sposta il puntatore di quello della tabella locale.
- Scheduling: Vista la sparsità delle tabelle, si usa un dinamic scheduling. Di default la block size è 1 e si può avere false sharing. La dimensione del blocco è stata settata a  $(64/\text{sizeof}(\text{Node*})) \times 16$ , cioè 16 cache line.



# Analisi

Confronto delle prestazioni tra versione sequenziale e parallela su dataset di diverse dimensioni.

- Dataset: da 100KB a 1.5 GB.
- Metrica: Speed-up  $T_{seq}/T_{par}$ .

# Analisi

## Testo

Esempi di 3-grams più frequenti trovati:

- **100KB:** This is the unique input size for which the most frequent 3-grams are *randallsville new york* (24), *variety of quartz* (10) and *in the uk* (9).
- **100MB - 1.5GB:** The most frequent 3-grams always the same: *one of the*, *as well as*, and *a lot of*. The frequencies in 1.5 GB file are respectively: 118445, 106364, and 64775.

# Analisi

## Hash Table

Fill factor  $\alpha = \frac{\text{busy\_buckets}}{\text{table\_length}}$ :

- **100 KB:** 0.0017.
- **100 MB:** 0.6853.
- **500 MB:** 0.991.
- **1 GB:** 0.9981.
- **1.5 GB:** 1.0000.

# Analisi

## Tempo - 100KB

Parallelo più lento per l'overhead del fork-join.

- Seq: 0.02s.
- Par: 0.057s.

# Analisi

## Tempo - 100MB

- Seq:
  - Totale: 3.33s
  - Popolazione: 3.06s
- Par:
  - Totale: 0.67s
  - Popolazione: 0.56s
- Speed-up:  $4.97\times$

# Analisi

## Tempo - 500MB

- Seq:
  - Totale: 31.38s
  - Popolazione: 29.72s
- Par:
  - Totale: 4.41s
  - Popolazione: 2.65s
- Speed-up:  $7.11\times$

# Analisi

## Tempo - 1GB

- Seq:
  - Totale: 53.79s
  - Popolazione: 51.03s
- Par:
  - Totale: 6.65s
  - Popolazione: 3.88s
- Speed-up:  $8.09\times$

# Analisi

## Tempo - 1.5GB

- Seq:
  - Totale: 207.66s
  - Popolazione: 199.02s
- Par:
  - Totale: 20.15s
  - Popolazione: 11.76s
- Speed-up:  $10.3\times$



# n-grams counting

**Edoardo Sarri**

Grazie



UNIVERSITÀ  
DEGLI STUDI  
FIRENZE